ETF Project Masterclass — Lesson 1

What Is an ETF, Really?

Foundational lesson before we touch the project itself. Goal: by the end of this, you can explain to Umesh — in your own words — what an ETF is, what NAV and market price mean, and why those two numbers can ever differ.

---

1. Start with a mutual fund

Forget ETFs for a second. A **mutual fund** is simple:

- Thousands of people each put in some money.
- That pooled money is used to buy a basket of things — stocks, bonds, gold, whatever the fund is designed to hold.
- A professional manager (an AMC — Asset Management Company, e.g. Mirae Asset) runs the basket.
- Each person owns a *slice* of the whole pool, called a **unit**.

If the pool is worth ₹100 crore and there are 10 crore units outstanding, each unit is worth ₹10. That per-unit value is the **NAV — Net Asset Value**. It's arithmetic, not opinion: `NAV = (value of everything the fund holds) / (number of units)`.

Traditionally, to buy or sell units of a mutual fund, you go to the AMC directly (or through a distributor), and you get that day's official NAV. There's no "market" for it — one price, set once a day, after the market closes.

2. An ETF is a mutual fund that also trades like a stock

An **ETF (Exchange-Traded Fund)** is the same idea — pooled money, a basket of holdings, a professional manager — with one twist: **its units are also listed on a stock exchange (NSE/BSE)**, so you can buy and sell them all day, the way you'd buy and sell shares of Reliance or TCS.

So now there are **two ways** to get in or out of an ETF:

Route	Who uses it	Price
Directly with the AMC ("primary market")	Mostly large players — big institutions, specialised market makers	That day's official NAV
On the exchange ("secondary market")	Everyone, including retail investors — anyone with a demat account	Whatever price buyers and sellers agree on, live, all day

This is the single most important structural fact about ETFs, and almost everything in our project hangs off it. Hold onto it — we'll come back to *why* two prices exist for the "same" thing in Lesson 2 (market makers and arbitrage).

3. What does an ETF actually hold?

Depends on the ETF — that's the whole point of the "basket" idea:

- A **Nifty 50 ETF** holds (roughly) the same 50 stocks as the Nifty 50 index, in the same proportions. Buying one unit is like buying a tiny slice of all 50 companies at once.
- A **Gold ETF** holds physical gold (or gold-backed instruments) in a vault. One unit ≈ a small amount of gold, without you having to buy, store or insure actual bars.
- A **Silver ETF** — same idea, with silver.

Our project is specifically about **Mirae's gold ETF (`GOLDETF`) and silver ETF (`SILVERAG`)**. Commodity ETFs (gold/silver) behave a bit differently from equity ETFs — we'll get into that — but the core mechanics (NAV vs market price) are the same.

4. NAV vs. market price — the two numbers that matter

- **NAV**: the official, once-a-day, "fair value per unit" published by the AMC (via AMFI), calculated from what the fund actually holds. Published after markets close.
- **Market price**: whatever price the ETF is trading at on the exchange, right now, because that's what a buyer and seller just agreed to.

In a healthy, well-functioning ETF, these two numbers stay very close to each other. Not identical, but close — within a fraction of a percent, most of the time. There's a mechanism that keeps them close (that's Lesson 2). When that mechanism works, the ETF is doing its job.

When they drift apart, we get a premium or a discount:

Term	Meaning	Simple example
Premium	Market price is *higher* than NAV	NAV = ₹145, market price = ₹150 → trading at a 3.4% premium
Discount	Market price is *lower* than NAV	NAV = ₹145, market price = ₹140 → trading at a 3.4% discount

The formula we'll use throughout the project (in basis points, where 100 bps = 1%):

```
premium (bps) = 10,000 × (market price − NAV) / NAV
```

Why does a premium matter to a real investor? If you buy an ETF at a 5% premium and the premium later disappears (price falls back toward NAV, which it almost always eventually does), you've lost ~5% for reasons that have nothing to do with gold or silver prices moving — purely because you overpaid relative to what the fund actually holds. This is the exact real-money risk our project is trying to warn people about *before* it happens, not after.

5. ⚠️ One important distinction to get right from day one: NAV vs. iNAV

This is worth being precise about now, because it's easy to blur the two and it will matter when you explain the project's evidence to Umesh.

- **NAV** (what we defined above): the *official*, end-of-day value. One number per fund per day.
- **iNAV** (indicative NAV): an *estimated, live, intraday* version of NAV that updates continuously while the market is open — a running guess at "what would NAV be right now, if we calculated it this second." It's published by the AMC/exchange but it is an estimate, not the official figure, and (importantly for us) **there's no free historical archive of iNAV data in India** — it only exists live, in the moment.

Why this matters for our project specifically: when you read that "silver ETFs traded 5–10% above NAV for five days in October 2025," that was mostly reported as a premium **over iNAV** (the live estimate), not over the official end-of-day NAV. Our model, because of what data actually exists and is usable, predicts the deviation from the **official NAV**, not

iNAV. These are related but not the same measurement — a bit like the difference between a shop's live "estimated wait time" and the time it actually took, printed on your receipt afterwards.

We'll handle this properly later (there's a clean fix: redefine what exactly we're testing the model against), but I'm flagging it now so you never describe the project to Umesh as "predicting the iNAV premium." It predicts the **close-price-vs-official-NAV** deviation. Keep that phrase in your back pocket.

6. What you should be able to say to Umesh after this lesson

In your own words, something like:

> "An ETF is a fund — a basket of assets, like gold or a set of stocks — that's professionally managed but also listed on the exchange, so it trades all day like a stock. It has an official NAV, the 'true' value per unit based on what it holds, published once a day. But because it also trades on the exchange, it has a separate live market price. Normally those two stay close together, but sometimes they drift apart — that's a premium or a discount. If someone buys during an inflated premium, they can lose money even if the underlying gold or silver didn't move at all, just because the premium corrected."

That one paragraph is a complete, correct, non-technical foundation. Everything else builds on it.

7. Questions Umesh might ask at this stage (and how to answer)

"Isn't an ETF basically a stock, then?" → "It trades like one, but legally it's a mutual fund scheme — it's backed by real holdings (gold, an index, bonds), and there's an official NAV behind it, which a stock doesn't have. A stock's 'fair value' is a matter of opinion; an ETF's NAV is arithmetic."

"Why would the market price ever NOT match the NAV? Isn't that arbitrage-proof?" → "There's normally a mechanism that keeps them aligned — that's exactly what Lesson 2 covers (market makers, creation/redemption). The project exists because that mechanism can break down under specific conditions, and that's what we're trying to forecast."

"Is this specific to gold/silver, or all ETFs?" → "The mechanics apply to all ETFs, but the project is scoped specifically to gold and silver ETFs, because that's where Mirae had two live, well-documented incidents in the last year, and where the dynamics are cleanest to study."

Stop here. Next lesson (whenever you're ready): *why* price and NAV normally stay together — market makers, creation/redemption, and the arbitrage loop — which sets up the actual mechanism our project is trying to monitor. Ask me anything about this lesson first, or just say "continue."

Lesson 2

Why Price and NAV Normally Stay Together — The Arbitrage Loop

Last lesson ended with a promise: there's a mechanism that keeps market price close to NAV. This lesson is that mechanism. It's the single most important piece of market mechanics for the whole project, because our project is about forecasting when this mechanism is about to fail.

---

1. The cast of characters

- **AMC** — Mirae Asset Investment Managers. Runs the fund, calculates NAV, holds the gold/silver in a vault.
- **Market Maker (MM)** — a special exchange member the AMC appoints to constantly quote both a buy price and a sell price for the ETF on the exchange, all day. Mirae's scheme documents require **at least two market makers per ETF**. Think of an MM as a professional middleman whose job is to always be willing to trade with you, so the ETF never just sits there with no buyer or seller available.
- **Large investors** — big players (institutions, large HNIs) who can also deal directly with the AMC, typically above a ₹25 crore threshold in Mirae's schemes.
- **You, the retail investor** — trade only on the exchange, with whoever's on the other side (often an MM).

2. The two ways to "create" or "destroy" ETF units

Here's the part that makes the magic work. An MM (or a large investor) can go to the AMC directly and do one of two things, **in bulk, in a big fixed batch called a "creation unit"**:

- **Creation**: Hand the AMC a basket of the underlying assets (e.g., the right amount of physical silver) → AMC gives back a big batch of brand-new ETF units, priced at that day's NAV.
- **Redemption**: Hand the AMC a big batch of ETF units → AMC gives back the underlying basket (or cash equivalent), again at NAV.

This only happens in large, fixed lot sizes — e.g., ICICI Prudential's Silver ETF creation unit is 30,000 units, Mirae's Defence ETF basket is 65,000 units. Retail investors never do this directly; only MMs and large investors do.

Crucially: this creation/redemption always happens at NAV, never at the market price. That's the lever that pulls market price back toward NAV whenever they drift apart.

3. Why this keeps the two prices glued together

This is the actual arbitrage logic — walk through it once and it'll stick:

Scenario: ETF is trading at a premium (market price ₹150, NAV ₹145)

1. An MM notices the exchange is paying ₹150 for something the AMC will hand out, in bulk, for the equivalent of ₹145.
2. The MM buys the underlying basket (silver), takes it to the AMC, gets new ETF units at NAV (₹145-equivalent each).
3. The MM sells those new units on the exchange at the market price (₹150).
4. Profit: ₹5 per unit, essentially risk-free, in bulk.
5. But step 3 — selling a large batch of units into the market — pushes the market price *down* (more supply). Step 2 pushes the *demand* for physical silver and new-unit supply up. This repeats until the premium shrinks to nothing (or to the small amount it costs to do all this).

Scenario: ETF is trading at a discount (market price ₹140, NAV ₹145) — the mirror image: MM buys cheap units on the exchange, redeems them at the AMC for the underlying basket worth more, sells the basket, pockets the difference. This buying pressure on the exchange pushes the market price back *up* toward NAV.

This self-correcting loop is why, most of the time, ETF price ≈ NAV. It's not regulation forcing it — it's profit-seeking MMs doing it naturally, every single day, automatically.

4. So why does it ever fail?

This is the whole reason our project exists. The loop above has several moving parts, and *any* of them can jam:

What can go wrong	Example from Mirae's own recent history
The physical basket can't be sourced fast enough	Oct 2025: a global silver supply squeeze meant MMs physically couldn't get enough silver to create new units fast enough. Premium stayed at 5–10% for five straight days because the "fix it" mechanism (step 2 above) was choked.
The AMC pauses creation	Sometimes an AMC stops accepting new creation requests (e.g., hitting a regulatory cap — more on this in a later lesson)
The NAV the MM would get is stale	If NAV is calculated at a time that doesn't reflect what's happening right now (gold/silver trade globally 24/7 but NAV is struck once, on an Indian-hours snapshot), the "₹145" the MM would receive isn't really ₹145 of current value — so the arbitrage math breaks down and MMs hesitate
MMs pull back in volatile/thin conditions	If markets are moving violently, MMs widen their quotes or step back to protect themselves — less of the correcting force is active
Circuit bands/halts physically block trading	If the exchange halts the price from moving at all (a "circuit"), price literally cannot converge to NAV even if everyone wants it to

Each of these is a *distinct, identifiable condition*. That's the insight the research document builds on: these aren't random, unknowable events — they tend to show early warning signs in things like trading volume, price momentum, and how peer ETFs (other gold/silver

ETFs) are behaving. **That's exactly the signal our ML model is trying to learn to recognise, a day ahead.**

5. One honest gap you should know about (and can mention to Umesh as a strength, not a weakness)

In the US, regulators publish data on exactly how much creation/redemption activity is happening each day — you can literally see the arbitrage mechanism at work. **In India, no such public data exists.** We can't directly see "how much silver did MMs create today." This means our model can't observe the mechanism directly — it has to infer stress in the mechanism *indirectly*, from things like price behaviour, trading volume, and peer ETFs. We're upfront about this limitation in the documents; it's a real constraint, not something to hide.

6. What you should be able to say to Umesh after this lesson

> "Market price and NAV stay together because of an arbitrage loop: authorised market makers can create or redeem ETF units directly with the AMC, in bulk, at NAV. If the market price drifts away from NAV, it's instantly profitable for an MM to trade against that gap — and doing so pushes the price back. That loop usually works perfectly. But it can jam — if the underlying asset can't be sourced fast enough, if the AMC pauses creations, if the reference NAV is stale, or if market makers pull back in volatile conditions. Our project is about predicting, a day ahead, when that loop is at risk of jamming for Mirae's gold and silver ETFs."

7. Questions Umesh might ask

"If arbitrage is supposed to fix this automatically, why do we need a forecasting model at all?" → "Because the fix isn't instant — it took five days in October 2025. The model isn't trying to replace the arbitrage mechanism, it's trying to give the sales team advance warning *while* the mechanism is under strain, so they can act before clients get caught overpaying."

"Who are Mirae's market makers for the gold/silver ETFs?" → Good one to actually ask him — the public scheme documents just say "at least two," without always naming them. This is a great real question to bring *to* him, not just answer.

"Can retail investors do creation/redemption too?" → "No — it's restricted to market makers and large investors, typically above a ₹25 crore threshold in Mirae's schemes. Retail only ever trades on the exchange, at market price."

Stop here. Next lesson: the three different "clocks" in this system — NAV, iNAV and market price — and the real regulatory events (the Oct 2025 silver squeeze, the Jan–Feb 2026 crash, and SEBI's new rulebook) that triggered this whole project. Say "continue" when ready, or ask me anything about creation/redemption first.

Lesson 3

Three Clocks, Price Bands, and the Real Events That Triggered This Project

Now we connect the mechanics to what actually happened. By the end of this lesson you'll be able to narrate, with dates and numbers, the three real episodes that justify why this project needs to exist at all.

---

1. Recap: three prices, three different "clocks"

Price	Who sets it	How often	The catch
Market price	The exchange, live trading	Continuously, 9:15 am–3:30 pm IST	Reacts instantly to any news, any time of day someone is trading
Official NAV	The AMC, via AMFI	Once per day, published after market close	A single daily snapshot — by the time you see it, it's already a few hours old
iNAV	AMC/exchange, an estimate	Updates continuously while the market is open (for equity ETFs, roughly every 15 seconds)	It's an *estimate* of what NAV would be right now — not official, and (important for us) **there's no historical archive of it in India**

2. Why three clocks is a real problem, not a technicality

Here's the catch that matters most for gold and silver: **gold and silver trade globally, 24 hours a day** (London, New York, etc.), but **the Indian ETF only trades 9:15 am–3:30 pm IST, and NAV is calculated once, at a specific valuation time tied to Indian market hours.**

So imagine silver jumps 5% overnight in London while the Indian market is closed. The next morning:

- The ETF's market price can react to that overnight move the moment trading opens.
- But the previous day's NAV doesn't know about it yet — it was calculated *before* the overnight move happened.

Result: a gap appears between market price and NAV that **isn't really mispricing at all — it's just that one clock updated and the other hasn't caught up yet.** Academic research on international ETFs (not India-specific, but well-established — Petajisto, 2017) makes exactly this point: a lot of "premium" on globally-traded-underlying ETFs is just timing, not inefficiency.

Why this matters for our project: our model has to be smart enough to not just flag "this ETF looks expensive" — it has to account for whether that gap is *genuine stress* (arbitrage loop jamming, as in Lesson 2) or *just the clock problem*. This is explicitly built into the feature design (more on that when we get to the technical architecture).

3. Price bands and "circuits" — the guardrail, and the trap

Exchanges don't let a stock or ETF move an unlimited amount in a single day — there's a **price band**, a ± limit around a reference price. If the price tries to move beyond that band, trading in that direction simply **locks** — no one can trade it any further until the band resets (usually the next day, or if the exchange intervenes).

Think of it like a dimmer switch with a hard stop: even if everyone in the room wants the lights much brighter, the switch physically won't turn further than its limit.

This is useful for stopping panic — but it has a dangerous side effect. If the band's *reference price* is stale (see clocks above), the band can lock the ETF at a price that's already wrong, while the true fair value keeps moving. Investors are then stuck — they can't sell, even though the "real" price has moved further. We'll see this happen for real in a moment.

The old system (until 7 Sep 2026): a fixed ±20% band, measured off the NAV from *two days earlier* ("T-2 NAV"). That lag is exactly what caused a problem in February 2026 — stay with me.

4. Episode 1 — The October 2025 silver premium (the "supply squeeze")

What happened: Starting 9 October 2025, silver ETFs across the industry traded **5–10% above their (indicative) NAV for five straight sessions.** At least eight asset management companies suspended new investments into silver ETF fund-of-funds because they couldn't source enough physical silver to keep up with demand (recall Lesson 2: the arbitrage loop needs the physical basket to be available — it wasn't).

- By 17 October, premiums collapsed back down — one major silver ETF actually closed *slightly below* NAV days later, showing how violently this can swing both ways once the pressure clears.
- **This is Archetype A from our documents: a physical supply squeeze.** It's also our project's primary validation case — the main real event we test our model against ("would it have flagged this coming, using only data available at the time?").

5. Episode 2 — The January–February 2026 crash (the "stale band" case)

What happened: Gold and silver hit a record peak on 29 January 2026, then **silver fell roughly 30% in a single day** globally. On Sunday, 1 February 2026 (a special trading session, held because of the Union Budget), Indian gold ETFs fell ~7% — but **silver ETFs mostly got stuck locked at the 20% lower circuit**, even though the actual decline that day was more like 10–15%. The band's reference price (T-2 NAV) was stale relative to how fast the underlying had actually moved.

- The exchange had to improvise — BSE issued a special circular *that same Sunday* moving the reference to a more current "T-1" NAV.
- This episode is **Archetype C: a crash combined with a band lock-up caused by a stale reference price** — a different failure mode from Episode 1, and our project's second validation case (a stress test, not the primary case).

6. SEBI's response — the new rulebook (effective 7 September 2026)

After these two episodes (and a public consultation in Feb 2026, where Mirae's own ETF product head was publicly quoted debating the proposed rules), the regulator — **SEBI**, India's securities regulator — rewrote the price-band framework for ETFs. Key changes, specifically for **gold/silver ETFs**:

	Old (until 4 Sep 2026)	New (from 7 Sep 2026)
Reference price	Stale, 2 days old ("T-2 NAV")	Fresher: last 30 minutes of trading the previous day
Band width	Fixed ±20%	Starts narrower (±6%), but can **flex outward in steps, with no upper cap**, if the move is genuine
Pre-market	No special mechanism	A new "call auction" session before the market opens, specifically for gold/silver ETFs

In plain terms: SEBI tried to fix exactly the stale-reference-price problem that caused Episode 2, by making the reference price fresher and letting the band move with genuine price action instead of just locking people out.

Why this matters to our project: this is a brand-new rule regime, live for less than two months as of today. Nobody has publicly studied how it's actually behaving yet. Our project doesn't try to simulate or predict the new band mechanics directly (that's deliberately parked as future work) — but it does treat "before vs. after 7 Sep 2026" as a labelled regime change, because the old data and the new data may behave differently, and pretending otherwise would be sloppy.

7. One more episode, just for context (not part of the core build)

Separately, in September 2026, Mirae's **international ETF** (tracking the NYSE FANG+ index, US tech stocks) was trading **21–33% above its NAV** — persistently, for months. This is a *different* failure mode entirely (Archetype D): it's not a temporary squeeze, it's a **regulatory cap** — SEBI limits how much Indian mutual funds can invest overseas in total, and once an AMC hits that cap, it can't create new units at all, so the "fix-it" arbitrage loop

(Lesson 2) is structurally switched off for as long as the cap binds. We mention this to Umesh as a real, acknowledged problem Mirae has — but it is **explicitly out of scope** for what we're building first; it's documented as a logical next extension.

8. What you should be able to say to Umesh after this lesson

> "There are really three prices in play — live market price, the official once-a-day NAV, and a live-but-unofficial iNAV estimate — and because gold and silver trade globally around the clock while the Indian ETF only trades Indian market hours, these clocks can get out of sync in ways that look like mispricing but are partly just timing. On top of that, exchanges cap how far price can move in a day with a 'band' — useful for stopping panic, but if the band's reference price is stale, it can lock investors in at the wrong price, which is exactly what happened to silver ETFs in February 2026. Mirae lived through two real episodes in the last year — the October 2025 supply-squeeze premium, and the February 2026 crash with band lock-ups — and SEBI just rewrote the rulebook in September 2026 partly in response. Our project uses those two real episodes as the evidence base and test cases for the forecasting model."

9. Questions Umesh might ask

"Didn't the new SEBI rules already fix this problem?" → "They fix the *band lock-up* mechanism — the fresher reference price should reduce stale-circuit cases going forward. But they don't fix the underlying cause of a premium in the first place — a physical supply squeeze (Episode 1) can still happen under the new rules. Our model is about forecasting *that*, not about the band mechanics, which we've deliberately kept out of this first version."

"Has anyone studied how the new rules are actually working since September?" → "Not publicly, as of now — it's only been live about two months. That's actually a gap our forward-testing approach could eventually help fill, but it's not the core deliverable."

"Why only these two episodes and not more?" → "These are the two best-documented, most recent, real events directly involving gold and silver ETFs — including Mirae's own silver ETF. Using real, dated, verifiable episodes rather than synthetic examples is a deliberate choice, so the evaluation means something."

Stop here. Next lesson: zooming in on Mirae specifically — its ETF lineup (`GOLDETF` , `SILVERAG` , and others), where Umesh's role sits, and exactly how this project is meant to create value for the sales team he runs. Say "continue" when ready, or ask anything about this lesson first.

Lesson 4

Mirae Asset, the ETF Lineup, and Where Umesh Fits

Now we zoom from "the market in general" to the specific company and the specific person you're pitching tomorrow.

---

1. Mirae Asset, quickly

Mirae Asset Investment Managers (India) is the AMC (fund house) — around 93 schemes across equity, debt, hybrid, ETF and index funds as of mid-2026. Passive investing (index funds + ETFs) is an explicit growth area for them, not a side business. They also own a broking arm (Mirae Asset Sharekhan), which is a built-in distribution channel.

2. The specific products this project is about

Product	Ticker	What it is
Mirae Asset Gold ETF	`GOLDETF`	Tracks the domestic price of gold
Mirae Asset Silver ETF	`SILVERAG` (changed from `SILVRETF` in Sep 2025 — a good detail to know, since old articles/data may still reference the old ticker)	Tracks the domestic price of silver

Product	Ticker	What it is
Silver/Gold ETF **Fund-of-Funds**	—	Funds that buy the ETF on your behalf — this is the product that got *paused* for new investment in October 2025, not the ETF itself. Useful nuance: the squeeze hit the FoF gateway, while the underlying ETF kept trading.

Scale context, so you can speak with numbers: gold and silver ETFs reportedly made up **54% of all of Mirae's ETF flows in February 2026** — they're not a side product, they're a huge and currently very active part of the business. (`54%` — remember this number, it's a strong, concrete line for the pitch.)

Mirae also has a separate **international ETF** (tracking the NYSE FANG+ index — big US tech stocks) that's been trading at a large, persistent premium due to a regulatory cap — this is the Episode 4/Archetype D we touched on last lesson, kept **out of scope** for this project but worth knowing it exists if Umesh brings it up.

3. Where Umesh sits

Umesh Kumar Daila is **Head of ETF Sales** — his public role, from conference appearances and interviews, centres on **national distribution of Mirae's ETFs**: getting distributors, advisors, platforms and institutions to sell and recommend Mirae's ETFs, and educating them on the products. He's publicly spoken about things like preferring gold over silver on risk-reward, and thematic ETFs as "satellite" allocations.

What this means for your pitch: Umesh's day job is not managing the fund or setting NAV — it's making sure the people who sell these ETFs to end clients have good information and good timing. That's exactly the audience your project is built for. Keep this front and centre: this isn't a trading model, it isn't a fund-management tool — it's a **decision-support tool for the sales conversation.**

4. The real pain point, stated the way Umesh would feel it

Think about what actually happens on the ground during an episode like October 2025:

- A distributor or advisor has been happily recommending the silver ETF to clients for months.
- Suddenly it's trading at a 7% premium, nobody on the sales side knew it was building, and a client who buys in now is set up to lose money the moment the premium corrects — **through no fault of the metal's price**, purely because they overpaid relative to NAV.
- When the correction happens, it's the advisor and the distributor who look bad to the client, and ultimately it reflects on Mirae's brand. The AMC's own response in Oct 2025 was reactive — several houses scrambled to pause FoF inflows only *after* the squeeze was already underway.

That reactive scramble is the gap this project targets: today, nobody on the sales side has an advance, data-backed signal that says "this one's getting risky, be careful recommending it right now." They find out the same way everyone else does — by watching the price.

5. The three concrete use cases this project actually builds (be precise here — don't overclaim)

The source documents describe eight possible use cases for a tool like this, but **this specific project deliberately builds only three of them** — it's a focused demonstration, not a platform:

1. **A daily risk list** — "is this ETF trading rich or cheap right now, and is stress building?" — something a salesperson could glance at before a client call.
2. **An evidence-backed "don't buy at a premium" narrative** — concrete, data-backed talking points for distributors, instead of a vague warning.
3. **A replay of the October 2025 and February 2026 episodes** — showing, with real dates and real numbers, what the system would have said *at the time*, proving the concept works on real history rather than just a plausible-sounding idea.

Everything else — automated alerts, a live production dashboard inside Mirae's systems, monitoring the international ETF premium, a GenAI assistant — is explicitly **future work**, documented but not built. Being upfront about this scoping is a strength, not a weakness: it shows discipline rather than over-promising.

6. What you should be able to say to Umesh

> "Mirae's gold and silver ETFs aren't a side product — they were over half of all ETF flows in February. This project builds a tool specifically for your side of the business: a daily risk list that flags, in advance, when `GOLDETF` or `SILVERAG` look like they're heading into an unusual premium or discount, plus the evidence to back up a 'don't recommend this right now' conversation with a distributor. I've deliberately kept the first version to three things — the risk list, the talking points, and a replay proving it would have worked on last October's real squeeze — rather than trying to build an entire platform."

7. Questions Umesh might ask

"Does this replace the FoF subscription-halt decision the AMC already makes?" → "No — that's a product/compliance decision made with internal information this project doesn't have access to. This is meant to support the sales conversation with earlier, data-backed warning — a complement, not a replacement."

"Would this have actually caught the October squeeze in advance, or only explained it afterwards?" → "That's the central test of the project — we deliberately built a backtest using only data that would have been available *at the time*, and check whether the model would have raised a flag before the 9 October premium became visible. If it didn't, I'll say so honestly rather than overstate the result."

"Why not include the international ETF premium problem? That's a bigger headache right now." → "Agreed it's a real and current issue — it's documented in the research as a clear next step. I kept the first build narrow and evidence-based on gold/silver specifically because that's where I have two clean, well-documented real events to validate against; the international-ETF problem is a structurally different mechanism (a regulatory cap, not a supply/demand stress) and deserves its own dedicated build."

Next: Lesson 5 — stating the actual problem and solution precisely (the one-sentence question the project answers, and why this needs AI/ML rather than a simple rule).

Lesson 5

The Problem, Stated Precisely — and Why This Needs Machine Learning at All

Before touching any technical architecture, you need to be able to state the exact question the project answers, in one breath, and defend why a simple rule isn't good enough.

---

1. The one-sentence question

> **"Which gold and silver ETFs are likely to trade at an unusually large premium or discount to NAV tomorrow, and how confident can we be?"**

Memorise this sentence. Everything in the project traces back to it. Notice three things packed into it:

- **"Tomorrow"** — it's a forecast, not a description of today's price (which anyone can already see on the exchange).
- **"Unusually large"** — not just any tiny wobble; we care about dislocations big enough to actually matter to an investor.
- **"How confident"** — the system doesn't just make a bold claim, it also says how sure it is. This turns out to be one of the most important design choices, and we'll spend real time on it.

2. Why can't you just use a simple rule? ("If premium > 5%, alert")

This is probably the single most likely sharp question Umesh (or anyone technical reviewing this later) could ask, so let's build the answer properly.

A simple fixed rule has three real problems:

1. **Different ETFs have different "normal."** One ETF might normally sit at a tiny 0.2% premium; another might normally run at 1.5% because it's thinner and harder to trade. A single fixed threshold (e.g., "5%") either misses the first ETF's real stress (which

might show up at 2%) or constantly cries wolf on the second. Our approach instead learns and tracks each ETF's **own recent normal** and flags *deviations from that*, not an arbitrary universal number.

2. **A rule reacts to today; it doesn't anticipate tomorrow.** "Premium is 6% right now, alert!" is not a forecast — it's a thermometer reading. By the time you see it, the opportunity to warn a distributor *before* a client buys in has already partly passed. The genuinely hard, useful problem is: can you see the pressure *building* — in volume, momentum, how peer ETFs are behaving — a day *before* the premium spikes? That's a pattern-recognition problem across many weak, interacting signals, which is exactly what machine learning is suited for and a fixed rule is not.
3. **A rule can't tell you how confident to be.** "Alert" vs "no alert" is a blunt, binary signal. A sales team making a real decision benefits enormously from knowing *how* likely a dislocation is — a cautious "maybe, 15% chance" calls for a different conversation than a confident "70% chance, something is clearly building." Getting a trustworthy probability (not just a guess dressed up as one) out of messy, limited financial data is itself a real technical problem — it's called **calibration**, and it's one of the project's central design goals (more in a later lesson).

3. The two things the model actually predicts

The project doesn't produce one single number — it produces two complementary outputs, each day, for each ETF:

	Plain description	Why it's useful separately
Task A — the premium range	"Tomorrow, we expect this ETF's premium to land somewhere between X and Y, most likely around Z."	Tells you the *expected level* — is it going to be mildly rich, or wildly rich?
Task B — the tail-risk probability	"There's a P% chance tomorrow's premium will be an unusual, out-of-character jump away from where this ETF normally sits."	Tells you *how abnormal* tomorrow is likely to be, relative to that specific ETF's own history — this is the actual early-warning signal

A quick analogy: think of a patient's temperature. Task A is like forecasting tomorrow's expected temperature reading ("probably around 37.2–37.8°C"). Task B is like forecasting the probability of a fever spike specific to that patient's own baseline (someone who normally runs a little warm needs a different threshold than someone who normally runs cool). You want both — the expected reading, *and* a flag for "this would be unusual for this particular patient."

4. Why predict a range, not a single number?

Financial forecasting of anything is noisy — nobody can predict tomorrow's exact premium to the decimal. A model that confidently spits out one precise number ("tomorrow's premium will be exactly 342 bps") is lying about its own certainty. Instead, the project predicts a **range** (e.g., "90% chance it lands somewhere between -0.8% and +8.1%") — honest about uncertainty, and still genuinely useful, because a wide range *itself* is informative ("we're not sure, but it could swing a lot" is a meaningful warning on its own).

5. What "honest" actually means here — the baseline requirement

One design principle worth highlighting to Umesh, because it signals rigor rather than hype: **the ML model has to beat simple baselines, out-of-sample, or the project reports that honestly as a negative result.** The baselines are things like "assume tomorrow's premium equals today's" (called *persistence*) and "flag it if it's unusually far from its own 60-day average." If the sophisticated model can't outperform these simple rules on data it hasn't seen during training, that's reported plainly, not hidden. This matters because it's very easy to build a flashy-looking ML system that's actually no better than a napkin calculation — the project is explicitly designed to catch itself out if that's the case.

6. What you should be able to say to Umesh

> "The question I'm answering is: which gold or silver ETFs are likely to trade at an unusually large premium or discount tomorrow, and how confident should we be? A fixed rule like 'alert above 5%' doesn't work well because different ETFs have different normal premium levels, it only reacts after the fact rather than warning in advance, and it can't tell you how confident to be. So the model learns each ETF's own normal behaviour, predicts a likely range for tomorrow's premium, and separately estimates the probability of an unusual jump away from that ETF's own baseline — and I've built in a requirement that it has to genuinely beat simple rule-of-thumb baselines, or I report that it didn't."

7. Questions Umesh might ask

"How accurate can this realistically be? Markets are unpredictable." → "That's exactly why I'm not promising a precise number — I'm promising a calibrated range and a probability, tested honestly against simple baselines and against two real past events. If it turns out daily public data genuinely can't predict this well, that's a real, useful finding too, and I'll report it as such rather than oversell the result."

"What happens when it's wrong?" → "Being 'wrong' for a probability forecast isn't the same as being wrong for a yes/no claim. If the model says '20% chance of an unusual move' and the unusual move happens, that's not necessarily a failure — a well-calibrated 20% probability event should happen about 1 in 5 times. What we check is whether the model's stated confidence actually matches reality over many predictions — that's the calibration testing I mentioned."

"Is this just reading the current premium and reacting to it?" → "No — reading today's premium is the easy part, anyone can see that live on the exchange already. The actual value-add is forecasting *tomorrow*, using patterns in volume, momentum, and peer-ETF behaviour today that tend to precede a premium spike — before it's visible on the price tag."

Next: Lesson 6 — the technical architecture, step one: where the data actually comes from, and the messy real-world problems in getting clean numbers out of Indian market data (this is where we start speaking Umesh's and your language at the same time).

Lesson 6

Technical Architecture, Step 1: Where the Data Comes From

We now switch gears into the build itself. This is where your CS background actually becomes an advantage — but we'll keep explaining the "why" in plain terms too, since this is the part you'd explain to Umesh as "here's what the system is built on."

---

1. The guiding rule: free, public, daily data only

A deliberate design decision: the whole project runs on **data anyone can download for free, once a day** — no paid data feeds, no insider access, no minute-by-minute trading data. This is a strength to state plainly to Umesh: *"This is something that could be rebuilt and extended with Mirae's own licensed data later, but the core concept is proven on data that's publicly available to anyone."*

2. The four data sources that actually matter for this build

Source	What it gives us	Real-world analogy
NSE/BSE end-of-day price file ("bhavcopy")	Every listed ETF's open, high, low, close price, and how much it traded, for every trading day	Like a daily newspaper's stock-price listing page, but as a structured file instead of print
AMFI's official NAV file	The official NAV for every mutual fund scheme (including every ETF), published once a day	The "true value per unit," as calculated by the fund house itself
Gold/silver global price + USD/INR exchange rate	Lets us build a rough, independent estimate of what the ETF's value "should" be, based on the actual metal price	A sanity-check reference point — are gold/silver themselves calm or volatile today?

Source	What it gives us	Real-world analogy
A small, hand-built table of each ETF's identity	ISIN (a unique ID), ticker history, inception date, issuer, metal type	Necessary because tickers *change* (Mirae's silver ETF changed from `SILVRETF` to `SILVERAG` in Sep 2025!) — without this, the pipeline could silently mix up old and new data for the same fund

What's deliberately NOT used: live intraday prices, the live iNAV estimate (recall Lesson 3 — no historical archive exists for it in India), and any order-flow / creation-redemption data (because, as covered in Lesson 2, that data simply isn't published in India at all). The project is explicit about these as real constraints, not oversights.

3. Why "just download the files" is harder than it sounds

This is a good section to have in your pocket if Umesh asks "how long could this realistically take to build" — because real-world data is messy in specific, documented ways:

- **The exchange changed its file format partway through 2024.** Two different "schemas" (layouts) exist across the project's history and have to be reconciled into one consistent internal format.
- **Tickers change.** As above — if you key your data purely on "SILVRETF", you'd silently lose the ETF's history the moment it became "SILVERAG". The fix: always identify a fund by its **ISIN** (a permanent unique code), never by its changeable ticker.
- **NAV history is rate-limited.** AMFI's official download only allows 90 days of history per request — so building years of history means many small, repeated downloads, done politely (not hammering their servers).
- **Special trading sessions exist.** Markets are normally closed Saturdays/Sundays — except when they're not (recall the special Sunday session on 1 February 2026, during the Union Budget crash). Any code that assumes "yesterday = previous calendar day" will quietly get this wrong unless it uses a proper trading calendar.
- **Thin-trading days are deceptive.** If almost nobody traded an ETF on a given day, its "closing price" might just be a stale leftover number from hours earlier, not a meaningful market price. The pipeline has to flag these days rather than trust them blindly.

4. "Point-in-time" — the single most important engineering discipline in the whole project

Here's a concept that's easy to get wrong and devastating if you do: **a model being tested on history must only ever see information that would genuinely have been available on that date, at that time** — never anything from the future, even by one day.

Why this matters so much: imagine testing the model on 9 October 2025 but accidentally letting it peek at the fact that premiums spiked badly five days later. The model would look brilliant in the backtest — and be completely useless in real life, because it cheated. This mistake is called **data leakage**, and it's one of the most common ways ML projects quietly produce fake, impressive-looking results that fall apart the moment they're used for real. The project builds in automatic tests specifically designed to catch this (deliberately feeding a future value into the pipeline and checking that it never leaks backward into a past prediction).

A subtlety worth knowing: NAV is published *after* the market closes for the day. So today's NAV can be used to help predict *tomorrow's* premium, but it could never have been used to predict *today's* premium in real time — the system respects that exact sequencing.

5. What you should be able to say to Umesh

> "The system runs entirely on free, publicly available daily data — exchange closing prices, the official AMFI NAV file, and gold/silver reference prices — deliberately avoiding any paid or insider data, so the concept is provable on data anyone could access. A lot of the real engineering effort goes into making that data trustworthy: handling ticker changes like the silver ETF's rename, reconciling two different file formats the exchange has used, respecting special trading sessions like the Budget-day Sunday session, and — most importantly — making absolutely sure the model, when being tested on past events, never gets to see information it wouldn't have actually had at the time. That last part is what makes a backtest honest rather than a self-fulfilling prophecy."

6. Questions Umesh might ask

"How far back does the data go?" → "Back to each ETF's listing date — but practically, the richest evidence is from around October 2025 onward, since that's when both validation events happened and when the data quality questions (ticker changes, format changes) are freshest and best-documented."

"Could this use Mirae's own internal data instead of public data?" → "Potentially, yes — and that's actually a natural next step, especially around order flow or creation/redemption activity, which simply isn't public in India at all. This version is deliberately built to prove the concept on data anyone can get, which also makes it safe to demonstrate without touching anything proprietary."

"Isn't daily data too slow? Markets move within the day." → "That's a real limitation and I say so explicitly — this version doesn't do intraday forecasting. It's scoped to 'will tomorrow be unusual,' using today's end-of-day picture. Intraday (using the live iNAV estimate) is flagged as a clear future extension once the groundwork here is proven."

Next: Lesson 7 — feature engineering: turning raw price/NAV numbers into the actual signals the model learns from (this is the heart of the 'AI' in the project, explained without assuming you know ML).

Lesson 7

Feature Engineering — Turning Raw Numbers Into Signals

This is genuinely the heart of the "AI" in the project. A machine learning model doesn't look at a chart the way a human does — it only sees a table of numbers. Feature engineering is the craft of deciding which numbers to hand it, so it has a fair chance of learning something real.

---

1. The core idea, in one line

For every ETF, every single day, the system computes about **35–45 numbers** — called **features** — that summarise "what's going on with this ETF right now." The model's whole job is to find patterns across these numbers that tend to show up *before* a dislocation, using the two real episodes (and all the quiet, normal days) as its training examples.

Think of it like a doctor's intake form before a diagnosis: you wouldn't just hand a doctor a single temperature reading — you'd give blood pressure, heart rate, recent symptoms, family history. Each one alone tells you a little; together, patterns emerge that a single number can't show. Our features are that intake form for an ETF.

2. The five groups of features

Group	Plain description	Example	Why it helps
F1 — Premium history	How has this ETF's own premium been behaving recently — trending up, flat, volatile?	"Premium has risen for 4 of the last 5 days"	Building pressure often shows up as a trend before it becomes a spike
F2 — Liquidity / trading activity	Is trading volume unusually high or low versus normal? Is the price moving in wide swings intraday?	"Today's trading volume is 3.2x its usual 20-day average"	A volume surge often accompanies (or precedes) real stress — recall the October 2025 squeeze involved unusually heavy demand
F3 — Underlying metal & volatility	How has gold/silver itself been moving globally, and is there a gap between the ETF's NAV and what the metal price suggests it should be?	"Silver moved 4% overnight while NAV hasn't updated yet"	Directly captures the 'stale clock' problem from Lesson 3
F4 — Peer comparison	How is this ETF behaving compared to other gold or silver ETFs right now?	"Peer silver ETFs are trading, on average, 180 bps above NAV"	Contagion — if a squeeze is industry-wide (like Oct 2025, which hit multiple fund houses), peers moving together is itself a strong signal

Group	Plain description	Example	Why it helps
F5 — Context	Calendar and situational facts: is it near a big gold-buying festival (Dhanteras/Diwali), is this ETF brand new and thinly traded, is there an active fund-halt event right now, which regulatory regime are we in?	"5 days until Dhanteras"	Demand (and therefore stress) isn't random — it has known seasonal and situational drivers in the Indian market specifically

3. A worked mini-example (illustrative, not real data)

Say it's a Tuesday in early October. For Mirae's silver ETF, the feature table might show something like:

- F1: premium has been climbing for 4 straight days (small, but consistent)
- F2: trading volume is 2.8x its normal average
- F3: global silver jumped overnight; the proxy-vs-NAV gap is unusually wide
- F4: three other silver ETFs are also showing rising premiums — this isn't isolated to Mirae
- F5: it's mid-October, a period of naturally higher festive gold/silver demand

None of these five facts alone proves anything. A volume spike alone could be nothing. A seasonal date alone means nothing most years. But **the combination — rising premium, unusual volume, a genuine supply-side gap, peer contagion, and a seasonally high-demand window, all at once** — is precisely the kind of multi-signal pattern a boosted-tree model (next lesson) is good at learning to weigh, far better than a human eyeballing five separate charts, or a rule that only checks one number.

4. "Reason codes" — making the model explainable, not a black box

A known weakness of ML models is that they can feel like an unexplainable black box: "the model says 24% risk" — but why? The project specifically avoids this by generating **plain-language reason codes** for every prediction, derived from which features actually drove that day's score — things like:

- "Turnover is 3.2x its 20-day average"
- "Peer silver ETFs are trading 180 bps above NAV"

This is a deliberate design choice, not an afterthought: a risk number with no explanation is nearly useless to a salesperson in a real conversation — they need to be able to say why an ETF looks risky, not just quote a percentage. This is also the single most "pitch-able" technical feature of the whole project for a business audience like Umesh — it turns a model output into something a non-technical person can act on and repeat to a distributor.

5. What you should be able to say to Umesh

> "Each day, for each ETF, the system builds a profile of roughly 40 numbers — how the premium has been trending, how trading volume compares to normal, how the underlying gold/silver price has moved globally, how similar ETFs are behaving, and situational context like upcoming festivals. No single number proves anything on its own, but the combination is what the model learns to recognise as 'this looks like conditions that preceded past dislocations.' And critically, every prediction comes with plain-language reasons attached — not just a risk score, so a salesperson can actually explain why an ETF looks risky, not just quote a number at a distributor."

6. Questions Umesh might ask

"How do you know which of these actually matters?" → "There's a specific test for exactly that — called an ablation study — where we remove each group of features one at a time and measure how much worse the model gets without it. That tells us, empirically, which signals are actually pulling their weight, rather than just assuming."

"Could this just be picking up on something obvious, like 'it's festival season'?" → "That's a fair challenge, and it's exactly why the model is tested against simple baselines — if a basic seasonal rule did just as well, the project would report that honestly rather than dress it up as sophisticated AI."

"What about rumours or news — does it read the news?" → "Not in this version — that's a documented future extension (using news and search-trend data), deliberately left out here to keep the first build provable on clean, structured numeric data rather than adding the complexity of text analysis."

Next: Lesson 8 — the models themselves: what LightGBM is, in plain terms, why it (not a flashy neural network) was chosen, and how the system turns feature tables into the two outputs from Lesson 5.

Lesson 8

The Models — What's Actually Learning, and Why Not a Flashy Neural Network

Here we open the "AI" box itself. Keep it simple with Umesh — he needs the concept and the justification, not the math.

---

1. The model: gradient-boosted decision trees (LightGBM)

The core model used is called **LightGBM** — a well-established, widely used technique that builds its prediction out of many simple **decision trees**.

What's a decision tree, in plain terms? It's a flowchart of yes/no questions. "Is today's trading volume more than 2x normal? → yes → Is the peer-group premium above 150 bps? → yes → risk is elevated." One tree alone is a crude rule. The trick ("boosting") is: build hundreds of small trees, one after another, where **each new tree is specifically trained to correct the mistakes of all the trees before it.** The final prediction is the combined vote of all of them. This tends to be extremely strong on exactly this kind of problem: a moderate number of numeric features, a tabular structure (rows = ETF-days, columns = features), and relatively few historical examples of the actual extreme events we care about.

2. Why not a neural network / deep learning / "real AI"?

This is a very likely question, because deep learning gets the public attention. The honest, defensible answer:

- **Deep learning needs a lot of data to be worth it.** Our universe is small (a handful of gold/silver ETFs) and genuinely extreme events are rare (we're working with maybe a handful of real dislocation episodes in the historical record). Neural networks tend to need far more examples to learn reliably — with this little data, they're more likely to memorise noise than learn a real pattern.

- **Our data is tabular, not sequential-in-the-way-neural-nets-excel-at (like raw text or images).** Boosted trees are specifically strong on exactly this kind of structured, numeric, tabular data — this is a well-established finding in the ML field, not a cut corner.
- **Boosted trees are explainable** (this connects directly to the reason codes from Lesson 7) — you can ask, after the fact, exactly which features drove a specific prediction. Neural networks are notoriously harder to explain this way.

One clean sentence for Umesh: *"I chose the tool that's actually proven to work best on this type of data and this amount of data — not the flashiest-sounding one. A sophisticated-sounding model that's wrong because it doesn't fit the problem isn't actually sophisticated."*

3. How the model produces the two outputs from Lesson 5

- **Task A (the premium range):** instead of training one model to predict one number, the system trains **several versions of the same model, each aimed at a different point in the range** — one predicts "the pessimistic 5th-percentile outcome," one predicts "the middle, most-likely outcome," one predicts "the optimistic 95th-percentile outcome," and so on. Stacking these together gives the full range (e.g., "90% chance it lands between -0.8% and +8.1%, most likely around +3.8%").
- **Task B (the tail probability):** a separate model is trained specifically to answer a yes/no question — "was tomorrow an unusual deviation from this ETF's own normal, yes or no?" — using the labelled history of past days as training examples. Rather than outputting a blunt yes/no, it's tuned to output an actual **calibrated probability** (more below).

4. "Calibration" — probably the single most important, least obvious concept

A model can output something that *looks* like a probability ("73% risk!") without that number actually meaning anything reliable. **Calibration is the discipline of making sure a stated probability matches real-world frequency.** If the model says "20% chance of an unusual move" on 100 different days, roughly 20 of those days should actually turn out to be unusual — not 5, and not 60. If that holds up, the probability is **calibrated**, and can genuinely be trusted and acted on. If it doesn't hold up, the number is just decoration.

Why this matters commercially, not just technically: a sales team making real decisions based on "24% risk" needs that 24% to actually mean something close to 1-in-4, or the whole tool becomes untrustworthy the first time it's "wrong" in a way that was actually expected. The project builds and specifically tests for this — it's not assumed, it's measured.

5. Reality-check: the model is also compared against dumb baselines, always

As covered in Lesson 5, every result is reported **alongside** how a simple rule-of-thumb would have done on the same data. If LightGBM can't beat "assume tomorrow looks like today" by a meaningful, statistically defensible margin, that is reported as a negative or weak result — the project doesn't get to quietly bury an underwhelming outcome.

6. What you should be able to say to Umesh

> "The core model is a well-established technique called gradient-boosted trees — essentially hundreds of simple decision rules combined together, each one correcting the mistakes of the ones before it. I specifically didn't reach for a flashy neural network, because with this much data and this few historical extreme events, boosted trees are the proven, better-suited choice, and they're also explainable — which is exactly what feeds the plain-language reasons I mentioned earlier. The model doesn't just output a risk score; it's specifically tested for 'calibration' — meaning if it says there's a 20% chance of something unusual, that should actually happen roughly 1 time in 5 across many predictions, not just be a number that sounds confident."

7. Questions Umesh might ask

"Is this 'real AI' or just statistics with a fancy name?" → "It's genuinely machine learning — the model learns the patterns itself from historical data rather than being hand-coded with fixed rules. But I'd push back gently on 'fancy' being the goal — the right tool here is the one proven to work on this kind of data, and that happens to be boosted trees, not a large neural network. I care more about it being right and explainable than about it sounding impressive."

"How do you know the probabilities can be trusted?" → "That's literally one of the formal things the project tests — calibration testing, where we check whether the model's stated confidence levels actually match how often those outcomes occur in reality, specifically held out from training so it's a fair test."

"What if the model just learns to always say 'low risk' since most days are calm?" → "That's a real risk with rare-event prediction, and it's specifically guarded against — the training process is adjusted for how rare these events are, and the model is explicitly tested on how well it performs on controls (calm, liquid ETFs that should almost never trigger an alert) versus on the two real stress episodes, where it needs to actually say something useful."

Next: Lesson 9 — how the system is actually tested (the backtest): walk-forward validation, the two event replays, and what "pass/fail" honestly looks like.